

Arrays

An array is a data structure that stores one or more data values having some relation among them, in a single variable. For example, if you want to store the marks of 10 students in a class, then instead of defining 10 different variables, its easy to define an array of 10 length.

Arrays in PHP behave a little differently than the arrays in C, as PHP is a dynamically typed language as against C which is a statically type language.

How to Define an Array in PHP?

In PHP, there are two ways to define an array: first using the `array()` function and second using the square brackets.

The `array()` Function

The built-in `array()` function uses the parameters given to it and returns an object of array type. One or more comma-separated parameters are the elements in the array.

`array(mixed ...$values): array`

Each value in the parenthesis may be either a singular value (it may be a number, string, any object or even another array), or a key-value pair. The association between the key and its value is denoted by the `"=>"` symbol.

Example

Here is the example to show how you can use `array()` function in your programs

```
<?php
$arr1 = array(10, "asd", 1.55, true);
$arr2 = array("one"=>1, "two"=>2, "three"=>3);
$arr3 = array(
    array(10, 20, 30),
    array("Ten", "Twenty", "Thirty"),
    array("physics"=>70, "chemistry"=>80, "maths"=>90)
);
print_r($arr3);
?>
```

Output

Following is the output of the above code

```
Array
(
    [0] => Array
        (
            [0] => 10
            [1] => 20
```

```

        [2] => 30
    )
[1] => Array
(
    [0] => Ten
    [1] => Twenty
    [2] => Thirty
)
[2] => Array
(
    [physics] => 70
    [chemistry] => 80
    [maths] => 90
)
)

```

Using Square Brackets []

Instead of the array() function, the comma-separated array elements may also be put inside the square brackets to declare an array object. In this case too, the elements may be singular values or a string or another array.

```

<?php
    $arr1 = [10, "asd", 1.55, true];
    $arr2 = ["one"=>1, "two"=>2, "three"=>3];
    $arr3 = [ [10, 20, 30],
              ["Ten", "Twenty", "Thirty"],
              ["physics"=>70, "chemistry"=>80, "maths"=>90] ];
?>

```

Example

In this example, **arr1** is an indexed array. However, var_dump() which displays the structured information of any object, shows that each value is having its index as its key.

```

<?php
    $arr1 = [10, "asd", 1.55, true];
    var_dump($arr1);
?>

```

Output

It will produce the following result

```

array(4) {
    [0]=>
    int(10)
    [1]=>
    string(3) "asd"
    [2]=>
    float(1.55)
    [3]=>
    bool(true)
}

```

```
}
```

Example

The same principle applies to a multi-dimensional index array, where each value in an array is another array.

```
<?php
    $arr1 = [
        [10, 20, 30],
        ["Ten", "Twenty", "Thirty"],
        [1.1, 2.2, 3.3]
    ];

    var_dump($arr1);
?>
```

Output

It will produce the following outcome

```
array(3) {
  [0]=>
  array(3) {
    [0]=>
    int(10)
    [1]=>
    int(20)
    [2]=>
    int(30)
  }
  [1]=>
  array(3) {
    [0]=>
    string(3) "Ten"
    [1]=>
    string(6) "Twenty"
    [2]=>
    string(6) "Thirty"
  }
  [2]=>
  array(3) {
    [0]=>
    float(1.1)
    [1]=>
    float(2.2)
    [2]=>
    float(3.3)
  }
}
```

Types of Arrays in PHP

There are three different kind of arrays and each array value is accessed using an ID which is called the array index.

- **Indexed Array:** An array which is a collection of values only is called an indexed array. Each value is identified by a positional index starting from "0". Values are stored and accessed in linear fashion.

```
For example - $fruits = ["Apple", "Banana", "Orange"];
```

- **Associative Array:** If the array is a collection of key-value pairs, it is called as an associative array. The key component of the pair can be a number or a string, whereas the value part can be of any type. Associative arrays store the element values in association with key values rather than in a strict linear index order.

```
$marks = ["Ankit" => 85, "Eisha" => 90, "Deepak" => 78];
```

- **Multi Dimensional Array** If each value in either an indexed array or an associative array is an array itself, it is called a multi dimensional array. Values are accessed using multiple indices.

```
$students = [  
    ["Ankit", 85, "A"],  
    ["Eisha", 90, "A+"],  
    ["Deepak", 78, "B"]  
];
```

NOTE Built-in array functions is given in function reference PHP Array Functions

It may be noted that PHP internally considers any of the above types as an associative array itself. In case of an indexed array, where each value has index, the index itself is its key. The `var_dump()` function reveals this fact.

Properties of Arrays in PHP

Here are some properties of array

- An array in PHP is an ordered map that associates values to keys.
- A PHP array can be used to implement different data structures such as a stack, queue, list (vector), hash table, dictionary, etc.
- The value part of an array element can be other arrays. This fact can be used to implement tree data structure and multidimensional arrays.
- PHP supports both indexed arrays (numeric keys) and associative arrays (string keys).
- PHP provides many built-in functions for array manipulation, such as `array_push()`, `array_pop()`, `array_merge()`, etc.

There are two ways to declare an array in PHP. One is to use the built-in array() function, and the other is to use a shorter syntax where the array elements are put inside square brackets.

Accessing the Array Elements

To access any element from a given array, you can use the array[key] syntax.

Example

For an indexed array, put the index inside the square bracket, as the index itself is anyway the key.

```
<?php
    $arr1 = [10, 20, 30];
    $arr2 = array("one"=>1, "two"=>2, "three"=>3);

    var_dump($arr1[1]);
    var_dump($arr2["two"]);
?>
```

Output

It will produce the following output

```
int(20)
int(2)
```

Associative Array

If each element in a PHP array is a key-value pair, such an array is called an **associative array**. In this type of array, each value is identified by its associated key and not an index. Associative arrays are used to implement data structures such as dictionary, maps, trees, etc.

In PHP, the "=>" symbol is used to establish association between a key and its value.

Syntax of Associative Array in PHP

Both the approaches of declaring an array - the array() function and the square bracket notation - can be used. The two syntaxes used to declare an associative array is given below.

Using the array() function

```
$arr1 = array(
    "Maharashtra"=>"Mumbai",
    "Telangana"=>"Hyderabad",
    "UP"=>"Lucknow",
    "Tamilnadu"=>"Chennai"
);
```

Using the square bracket notation []

```
$arr2 = ["Maharashtra"=>"Mumbai",
    "Telangana"=>"Hyderabad",
    "UP"=>"Lucknow",
    "Tamilnadu"=>"Chennai"];
```

Structure of the Associative Array

If we call the var_dump() function, both the above arrays will show the similar structure

```
array(4) {
    ["Maharashtra"]=>
    string(6) "Mumbai"
    ["Telangana"]=>
    string(9) "Hyderabad"
    ["UP"]=>
    string(7) "Lucknow"
    ["Tamilnadu"]=>
    string(7) "Chennai"
}
```

The **key** part of each element in an associative array can be any number (integer, float or Boolean), or a string. The **value** part can be of any type. However, the float key is cast to an integer. So, a Boolean true/false is used as "1" or "0" as the key.

Basic Example of Associative Array

Take a look at the following example

```
<?php
    $arr1 = array(
        10=>"hello",
        5.75=>"world",
        -5=>"foo",
        false=>"bar"
    );
    var_dump($arr1);
?>
```

Output

It will produce the following output

```
array(4) {
    [10]=>
    string(5) "hello"
    [5]=>
    string(5) "world"
    [-5]=>
    string(3) "foo"
    [0]=>
    string(3) "bar"
}
```

Note that the key 5.75 gets rounded to 5, and the key "true" is reflected as "0". If the same key appears more than once in an array, the key-value pair that appears last will be retained, discarding the association of the key with earlier value.

PHP internally treats even an indexed array as an associative array, where the index is actually the key of the value. It means the value at the 0th index has a key equal to "0", and so on. A `var_dump()` on an indexed array also brings out this characteristics of a PHP array.

Iterating an Associative Array

A **foreach** loop is the easiest and ideal for iterating through an associative array, although any other type of loop can also be used with some maneuver.

Let us look at the **foreach** loop implementation, with each key value pair unpacked in two variables.

```
<?php
    $capitals = array(
        "Maharashtra"=>"Mumbai",
        "Telangana"=>"Hyderabad",
        "UP"=>"Lucknow",
        "Tamilnadu"=>"Chennai"
    );
```

```

        foreach ($capitals as $k=>$v) {
            echo "Capital of $k is $v \n";
        }
?>

```

Output

It will produce the following output

```

Capital of Maharashtra is Mumbai
Capital of Telangana is Hyderabad
Capital of UP is Lucknow
Capital of Tamilnadu is Chennai

```

Using foreach with array_search()

There is another way of using the **foreach** loop in PHP, where each element is stored in a variable. We can then separate the key and value parts using **array_search()** and use them in the loop body.

```

<?php
    $capitals = array(
        "Maharashtra"=>"Mumbai",
        "Telangana"=>"Hyderabad",
        "UP"=>"Lucknow",
        "Tamilnadu"=>"Chennai"
    );

    foreach ($capitals as $pair) {
        $cap = array_search($pair, $capitals);
        echo "Capital of $cap is $capitals[$cap] \n";
    }
?>

```

Output

It will produce the following output

```

Capital of Maharashtra is Mumbai
Capital of Telangana is Hyderabad
Capital of UP is Lucknow
Capital of Tamilnadu is Chennai

```

To use **for**, **while** or **do-while** loop, we have to first get the array of all the keys (use **array_keys()**), find the size and use it as the test condition in the loop syntax.

Using for loop with array_keys()

Here is how we can use a **for** loop to traverse an associative array

```

<?php
    $capitals = array(
        "Maharashtra"=>"Mumbai",
        "Telangana"=>"Hyderabad",

```

```

        "UP"=>"Lucknow",
        "Tamilnadu"=>"Chennai"
    );
    $keys=array_keys($capitals);

    for ($i=0; $i<count($keys); $i++){
        $cap = $keys[$i];
        echo "Capital of $cap is $capitals[$cap] \n";
    }
?>

```

Output

It will produce the following output

```

Capital of Maharashtra is Mumbai
Capital of Telangana is Hyderabad
Capital of UP is Lucknow
Capital of Tamilnadu is Chennai

```

Accessing Values Using Keys

In an associative array, the key is the identifier of value instead of index. Hence, to fetch value associated with a certain key, use `$arr[key]` syntax. The same can be used to update the value of a certain key.

In the following code, an associative array **\$arr1** is declared. Another array **\$arr2** is created such that it stores each pair from **\$arr1** with the value of each key being doubled.

```

<?php
    $arr1 = array("a"=>10, "b"=>20, "c"=>30, "d"=>40);
    foreach ($arr1 as $k=>$v){
        $arr2[$k] = $v*2;
    }
    print_r($arr2);
?>

```

Output

It will produce the following output

```

Array
(
    [a] => 20
    [b] => 40
    [c] => 60
    [d] => 80
)

```

The **print_r()** function used here displays the data stored in the array in an easy to understand human readable form.

Check if a Key Exists in an Associative Array

In the following example, we are showing how you can loop for a key in an associative array. Before accessing a key, verify that it exists by calling `array_key_exists()` or `isset()`.

```
<?php
    $arr = ["a" => 10, "b" => 20];

    if (array_key_exists("a", $arr)) {
        echo "Key 'a' exists!";
    }
?>
```

Output

Following is the output of the above code

Key 'a' exists!

Remove Elements from an Associative Array

In the following example, we are using `unset()` function which can remove a specific key-value pair from an associative array.

```
<?php
    $arr = ["a" => 10, "b" => 20, "c" => 30];
    unset($arr["b"]);
    print_r($arr);
?>
```

Output

Below is the output of the above code

Array ([a] => 10 [c] => 30)

Indexed Array

In PHP, the array elements may be a collection of key-value pairs or it may contain values only. If the array consists of values only, it is said to be an indexed array, as each element is identified by an incrementing index, starting with "0".

Creating an Indexed Array

An indexed array in PHP may be created either by using the `array()` function or with the square bracket **syntax**.

```
$arr1 = array("a", 10, 9.99, true);  
$arr2 = ["a", 10, 9.99, true];
```

Structure of an Indexed Array

Each element in the array has a positional index, the first element being at index "0". The `var_dump()` function reveals the structured information of these arrays as

```
array(4) {  
    [0]=>  
    string(1) "a"  
    [1]=>  
    int(10)  
    [2]=>  
    float(9.99)  
    [3]=>  
    bool(true)  
}
```

We can use the index to traverse the array, fetch the value at a given index or modify the value of an element in place.

Traversing an Indexed Array in PHP

Any type of PHP loop can be employed to traverse an array. If we want to use a **for** or **while** loop, we have to find the number of elements in the array with `count()` function and use its value as the test condition for the counted **for** or **while** loop.

Example

The following code uses a **for** loop to list all the elements in an indexed array.

```
<?php  
    $numbers = array(10, 20, 30, 40, 50);  
  
    for ($i=0; $i<count($numbers); $i++){  
        echo "numbers[$i] = $numbers[$i] \n";  
    }  
?>
```

It will produce the following **output**

```
numbers[0] = 10
numbers[1] = 20
numbers[2] = 30
numbers[3] = 40
numbers[4] = 50
```

You can also use a **while** or **do-while** loop to traverse an indexed array. Here too, we need to find the array length with count() function.

Example

The following code traverses the given indexed array in reverse order

```
<?php
    $numbers = array(10, 20, 30, 40, 50);
    $i = count($numbers)-1;
    while ($i>=0){
        echo "numbers[$i] = $numbers[$i] \n";
        $i--;
    }
?>
```

It will produce the following **output**

```
numbers[4] = 50
numbers[3] = 40
numbers[2] = 30
numbers[1] = 20
numbers[0] = 10
```

Accessing the Array Elements Using Index

You can access any value from an array using the **array[index]** syntax. The value at a specific index may be assigned with a new value. The array is thus modified in place.

Example

The following program fetches the values from an array **\$arr1** and places them in **\$arr2** in the reverse order. So the value at 0th position in \$arr1 becomes the last value in \$arr2.

```
<?php
    $arr1 = array(10, 20, 30, 40, 50);
    $size = count($arr1);

    for ($i=0; $i<$size; $i++){
        $arr2[$size-$i-1] = $arr1[$i];
    }

    for ($i=0; $i<$size; $i++){
        echo "arr1[$i] = $$arr1[$i] arr2[$i] = $$arr2[$i] \n";
    }
?>
```

It will produce the following **output**

```
arr1[0] = $10 arr2[0] = $50
arr1[1] = $20 arr2[1] = $40
arr1[2] = $30 arr2[2] = $30
arr1[3] = $40 arr2[3] = $20
arr1[4] = $50 arr2[4] = $10
```

Traversing an Indexed Array Using "foreach" Loop

You can also use the **foreach** loop to iterate through an indexed array. Take a look at the following **example**

```
<?php
    $arr1 = [10, 20, 30, 40, 50];
    foreach ($arr1 as $val){
        echo "$val \n";
    }
?>
```

It will produce the following **output**

```
10
20
30
40
50
```

Note that PHP internally treats the indexed array as an associative array, with the index being treated as the key. This fact can be verified by the `var_dump()` output of the array.

Example

We can unpack each element of an indexed array in the key and value variables with **foreach** syntax

```
<?php
    $arr1 = [10, 20, 30, 40, 50];
    foreach ($arr1 as $key => $val){
        echo "arr1[$key] = $val \n";
    }
?>
```

It will produce the following **output**

```
arr1[0] = 10
arr1[1] = 20
arr1[2] = 30
arr1[3] = 40
arr1[4] = 50
```

In PHP, an array may be a mix of only values and key-value pairs. PHP assigns the index only to the values without keys.

Example

In this example, PHP assigns incrementing index to the numbers, skipping the key-value pair appearing in it.

```
<?php
    $arr1 = [10, 20,
            "vals" => ["ten", "twenty"],
            30, 40, 50];

    var_dump($arr1);
?>
```

It will produce the following **output**

```
array(6) {
  [0]=>
  int(10)
  [1]=>
  int(20)
  ["vals"]=>
  array(2) {
    [0]=>
    string(3) "ten"
    [1]=>
    string(6) "twenty"
  }
  [2]=>
  int(30)
  [3]=>
  int(40)
  [4]=>
  int(50)
}
```

Properties of Indexed Arrays

Here are some of the properties of the indexed arrays are listed

- The initial element has an index of zero and grows sequentially.
- An indexed array can store a wide range of values, like texts, numbers, floats and booleans.
- Arrays in PHP can grow and decrease dynamically without being explicitly declared.
- Indexed arrays can be easily looped using for or foreach loops.
- PHP has several built-in functions for managing indexed arrays, like `count($arr)`, `array_push($arr, "Value")`, and `array_pop($arr)`.

Multidimensional Array

A multidimensional array is an array of arrays. In a PHP array, each element can be another array. If the array consists of values or key-value pairs with values being of singular scalar types, it is a one-dimensional array. If each element in an array is an array of one or more scalar values, it is a two-dimensional array.

A PHP array may be a two-dimensional associative array also, where each element of the outer array is key-value pair, the value being another associative array.

Syntax

Here is the Syntax for one dimensional indexed array

```
# one dimensional indexed array
$arr = [10, 20, 30, 40];
```

Following is the Syntax for one dimensional associative array

```
# one dimensional associative array
$arr = ["key1" => "val1", "key2" => "val2", "key3" => "val3"];
```

Here is the Syntax for two dimensional indexed array

```
# two dimensional indexed array
$arr = [
    [1,2,3,4],
    [10, 20, 30, 40],
    [100, 200, 300, 400]
];
```

Below is the Syntax for two dimensional associative array

```
# two dimensional associative array
$arr = [
    "row1" => ["key11" => "val11", "key12" => "val12", "key13" => "val13"],
    "row2" => ["key21" => "val21", "key22" => "val22", "key23" => "val23"],
    "row3" => ["key31" => "val31", "key32" => "val32", "key33" => "val33"]
];
```

Iterating over a 2D Array

Two nested loops will be needed to traverse all the elements in a 2D array. The **foreach** loop is more suitable for array traversal. A 2D array is like a tabular representation of data in rows and columns.

Traversing Indexed 2D Array

The following example shows how you can reproduce a 2D array in a tabular form

```
<?php
```

```

$tbl = [
    [1,2,3,4],
    [10, 20, 30, 40],
    [100, 200, 300, 400]
];
echo ("\n");
foreach ($tbl as $row){
    foreach ($row as $elem){
        $val = sprintf("%5d", $elem);
        echo $val;
    }
    echo "\n";
}
?>

```

Output

It will produce the following output

```

 1      2      3      4
10     20     30     40
100    200    300    400

```

Traversing Associative 2D Array

We can also employ two nested **foreach** loops to traverse a 2D associative array. Unpack each row of the outer array in row-key and row-value variables and traverse each row elements with the inner **foreach** loop.

```

<?php
$tbl = [
    "row1" => ["key11" => "val11", "key12" => "val12", "key13" => "val13"],
    "row2" => ["key21" => "val21", "key22" => "val22", "key23" => "val23"],
    "row3" => ["key31" => "val31", "key32" => "val32", "key33" => "val33"]
];

echo ("\n");
foreach ($tbl as $rk=>$rv){
    echo "$rk\n";
    foreach ($rv as $k=>$v){
        echo "$k => $v ";
    }
    echo "\n";
}
?>

```

Output

It will produce the following output

```

row1
key11 => val11  key12 => val12  key13 => val13
row2
key21 => val21  key22 => val22  key23 => val23
row3
key31 => val31  key32 => val32  key33 => val33

```

Accessing the Elements in a 2D Array

The `$arr[$key]` syntax of accessing and modifying an element in the array can be extended to a 2D array too. For a 2D indexed array, the *j*th element in the *i*th row can be fetched and assigned by using the expression "`$arr[$i][$j]`".

Indexed 2D Array

```
<?php
    $tbl = [[1,2,3,4], [10, 20, 30, 40], [100, 200, 300, 400]];

    # prints number in index 2 of the row 2
    print ("Value at [2], [2] :" . $tbl[2][2]);
?>
```

Output

It will produce the following output

```
Value at [2], [2] :300
```

Similarly, the value at *i*th row and *j*th column may be set to another value.

```
$tbl[2][2] = 250;
```

Associative 2D Array

If it is a 2D associative array, we need to use the row key and key-value variables of the desired column to access or modify its value.

```
<?php
    $tbl = [
        "row1" => ["key11" => "val11", "key12" => "val12", "key13" => "val13"],
        "row2" => ["key21" => "val21", "key22" => "val22", "key23" => "val23"],
        "row3" => ["key31" => "val31", "key32" => "val32", "key33" => "val33"]
    ];

    print "value at row2 - key22 is " . $tbl["row2"]["key22"];
?>
```

Output

It will produce the following output

```
value at row2 - key22 is val22
```

Multi-dimensional Array

In the above example, we had an array in which the associated value of each key was another collection of key-value pairs, and we call it as a 2D array. The concept can be extended to any number of levels. For example, if each element in the inner array associates its key to another array, it becomes a three-dimensional array.

Declaring a 3D Array

Here is an **example** of a three-dimensional array

```
$arr3D = [  
    [  
        [1, 0, 9],  
        [0, 5, 6],  
        [1, 0, 3]  
    ],  
    [  
        [0, 4, 6],  
        [0, 0, 1],  
        [1, 2, 7]  
    ],  
];
```

Traversing a 3D Array

To traverse such a 3D array, we need three nested **foreach** loops, as shown below

```
<?php  
$arr3D = [  
    [[1, 0, 9],[0, 5, 6],[1, 0, 3]],  
    [[0, 4, 6],[0, 0, 1],[1, 2, 7]],  
];  
  
foreach ($arr3D as $arr) {  
    foreach ($arr as $row) {  
        foreach ($row as $element) {  
            echo "$element ";  
        }  
        echo "\n";  
    }  
    echo "\n";  
}  
?>
```

Output

It will produce the following output

```
1 0 9  
0 5 6  
1 0 3  
  
0 4 6  
0 0 1  
1 2 7
```

However, it is entirely possible to declare an array extending upto any number of dimensions. For that we need to have a generalized solution to traverse an array of any dimensions.

Recurse Traversal of Multidimensional Array

The following code shows a recursive function that calls itself if the value of a certain key is another array. If we pass any array as an argument to this function, it will be traversed, showing all the k-v pairs in it.

```
function showarray($arr) {
    foreach ($arr as $k=>$v) {
        if (is_array($v)) {
            showarray($v);
        } else {
            echo "$k => $v ";
        }
    }
    echo "\n";
}
```

Example

Let us pass the above 3D array **\$arr3D** to it and see the result

```
<?php
$arr3D = [
    [[1, 0, 9],[0, 5, 6],[1, 0, 3]],
    [[0, 4, 6],[0, 0, 1],[1, 2, 7]],
];

function showarray($arr){
    foreach ($arr as $k=>$v){
        if (is_array($v)){
            showarray($v);
        } else {
            echo "$k => $v ";
        }
    }
    echo "\n";
}
showarray($arr3D);
?>
```

Output

It will produce the following output

```
0 => 1  1 => 0  2 => 9
0 => 0  1 => 5  2 => 6
0 => 1  1 => 0  2 => 3
0 => 0  1 => 4  2 => 6
0 => 0  1 => 0  2 => 1
0 => 1  1 => 2  2 => 7
```

This recursive function can be used with any type of array, whether indexed or associative, and of any dimension.

Example

Let us use a 2D associative array as argument to showarray() function

```
<?php
    $tbl = [
        "row1" => ["key11" => "val11", "key12" => "val12", "key13" => "val13"],
        "row2" => ["key21" => "val21", "key22" => "val22", "key23" => "val23"],
        "row3" => ["key31" => "val31", "key32" => "val32", "key33" => "val33"]
    ];

    function showarray($arr){
        foreach ($arr as $k=>$v){
            if (is_array($v)){
                showarray($v);
            } else {
                echo "$k => $v ";
            }
        }
        echo "\n";
    }
    showarray($tbl);
?>
```

Output

It will produce the following output

```
key11 => val11  key12 => val12  key13 => val13
key21 => val21  key22 => val22  key23 => val23
key31 => val31  key32 => val32  key33 => val33
```